

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL 11
MULTILIST**



Disusun Oleh :

NAMA : RAIHAN DZAKY MUFLIH

NIM : 103112430029

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Dasar Teori

Dasar teori dari praktikum struktur data ini berfokus pada pemahaman konsep Linked List sebagai salah satu struktur data dinamis yang memungkinkan penyimpanan dan pengelolaan data secara fleksibel. Circular Linked List merupakan variasi dari linked list di mana node terakhir menunjuk kembali ke node pertama, sehingga membentuk struktur melingkar tanpa ujung. Konsep ini memungkinkan proses traversal yang lebih efisien pada beberapa aplikasi tertentu. Setiap node dalam list terdiri dari data (info) dan pointer (next) yang menghubungkan node satu dengan node lain menggunakan alokasi memori dinamis. Selain itu, Multilist merupakan pengembangan dari konsep dasar linked list yang membentuk hubungan antara data induk dan data anak, sehingga cocok digunakan untuk data bertingkat atau bersifat hierarki. Dalam implementasinya, kita menggunakan operasi dasar seperti alokasi dan dealokasi memori, penyisipan elemen di awal, tengah, atau akhir list, penghapusan elemen, pencarian data berdasarkan kunci tertentu, serta pencetakan isi list. Teori ini menjadi fondasi penting dalam memahami bagaimana struktur data bekerja secara internal dan bagaimana hubungan antar-node dapat dimanfaatkan untuk menyelesaikan berbagai permasalahan dalam pemrograman.

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

```
#include <iostream>
#include <string>
using namespace std;

struct ChildNode {
    string info;
    ChildNode *next;
    ChildNode *prev;
};

struct ParentNode {
    string info;
    ChildNode *childHead;
    ParentNode *next;
    ParentNode *prev;
};

ParentNode *createParent(string info) {
    ParentNode *newNode = new ParentNode;
    newNode->info = info;
    newNode->childHead = NULL;
```

```

    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

ChildNode *createChild(string info) {
    ChildNode *newNode = new ChildNode;
    newNode->info = info;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}

void insertParent(ParentNode *&head, string info) {
    ParentNode *newNode = createParent(info);
    if (head == NULL) {
        head = newNode;
    } else {
        ParentNode *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->prev = temp;
    }
}

void insertChild(ParentNode *head, string parentInfo, string
childInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }

    if (p != NULL) {
        ChildNode *newChild = createChild(childInfo);
        if (p->childHead == NULL) {
            p->childHead = newChild;
        } else {
            ChildNode *c = p->childHead;
            while (c->next != NULL) {
                c = c->next;
            }
        }
    }
}

```

```

        c->next = newChild;
        newChild->prev = c;
    }
}

void printAll(ParentNode *head) {
    while (head != NULL) {
        cout << head->info;
        ChildNode *c = head->childHead;
        while (c != NULL) {
            cout << " -> " << c->info;
            c = c->next;
        }
        cout << endl;
        head = head->next;
    }
}

void updateParent(ParentNode *head, string oldInfo, string
newInfo) {
    ParentNode *p = head;
    while (p != NULL) {
        if (p->info == oldInfo) {
            p->info = newInfo;
            return;
        }
        p = p->next;
    }
}

void updateChild(ParentNode *head, string parentInfo, string
oldChildInfo, string newChildInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }

    if (p != NULL) {
        ChildNode *c = p->childHead;
        while (c != NULL) {
            if (c->info == oldChildInfo) {
                c->info = newChildInfo;
            }
        }
    }
}

```

```

        return;
    }
    c = c->next;
}
}

void deleteChild(ParentNode *head, string parentInfo, string
childInfo) {
    ParentNode *p = head;
    while (p != NULL && p->info != parentInfo) {
        p = p->next;
    }

    if (p != NULL) {
        ChildNode *c = p->childHead;
        while (c != NULL) {
            if (c->info == childInfo) {
                if (c == p->childHead) {
                    p->childHead = c->next;
                    if (p->childHead != NULL) {
                        p->childHead->prev = NULL;
                    }
                } else {
                    c->prev->next = c->next;
                    if (c->next != NULL) {
                        c->next->prev = c->prev;
                    }
                }
                delete c;
                return;
            }
            c = c->next;
        }
    }
}

void deleteParent(ParentNode *&head, string info) {
    ParentNode *p = head;
    while (p != NULL) {
        if (p->info == info) {
            ChildNode *c = p->childHead;
            while (c != NULL) {

```

```

        ChildNode *tempC = c;
        c = c->next;
        delete tempC;
    }

    if (p == head) {
        head = p->next;
        if (head != NULL) {
            head->prev = NULL;
        }
    } else {
        p->prev->next = p->next;
        if (p->next != NULL) {
            p->next->prev = p->prev;
        }
    }
    delete p;
    return;
}
p = p->next;
}
}

int main() {
    ParentNode *list = NULL;

    insertParent(list, "Parent A");
    insertParent(list, "Parent B");
    insertParent(list, "Parent C");

    cout << "\nSetelah InsertParent: " << endl;
    printAll(list);

    insertChild(list, "Parent A", "Child A1");
    insertChild(list, "Parent A", "Child A2");
    insertChild(list, "Parent B", "Child B1");

    cout << "\nSetelah InsertChild: " << endl;
    printAll(list);

    updateParent(list, "Parent B", "Parent B*");
    updateChild(list, "Parent A", "Child A1", "Child A1*");
}

```

```

    cout << "\nSetelah Update: " << endl;
    printAll(list);

    deleteChild(list, "Parent A", "Child A2");
    deleteParent(list, "Parent C");

    cout << "\nSetelah Delete: " << endl;
    printAll(list);

    return 0;
}

```

Screenshots Output

```

Setelah InsertParent:
Parent A
Parent B
Parent C

Setelah InsertChild:
Parent A -> Child A1 -> Child A2
Parent B -> Child B1
Parent C

Setelah Update:
Parent A -> Child A1* -> Child A2
Parent B* -> Child B1
Parent C

Setelah Delete:
Parent A -> Child A1*
Parent B* -> Child B1

[Done] exited with code=0 in 1.681 seconds

```

Deskripsi:

Program ini digunakan untuk mengelola sebuah daftar parent yang masing-masing dapat memiliki daftar child. Setiap parent dan child disimpan dalam bentuk node yang saling terhubung. Program menyediakan fitur untuk menambahkan parent baru, menambahkan child pada parent tertentu, memperbarui nama parent atau child, menghapus child tertentu, serta menghapus parent beserta seluruh child yang dimilikinya. Setelah setiap operasi, program menampilkan seluruh isi daftar agar perubahan yang terjadi dapat terlihat dengan jelas. Secara sederhana, program ini berfungsi sebagai sistem pengelolaan data parent-child yang dapat ditambah, diubah, ditampilkan, dan dihapus sesuai

kebutuhan.

- C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

#multilist.h

```
#ifndef MULTILIST_H_INCLUDED
#define MULTILIST_H_INCLUDED

#include <stddef.h>
#define Nil NULL

typedef int infotypeanak;
typedef int infotypeinduk;
typedef int boolean;
#define true 1
#define false 0

struct elemen_list_anak;
struct elemen_list_induk;

typedef struct elemen_list_induk *address;
typedef struct elemen_list_anak *address_anak;

struct elemen_list_anak {
    infotypeanak info;
    address_anak next;
    address_anak prev;
};

struct listanak {
    address_anak first;
    address_anak last;
};

struct elemen_list_induk {
    infotypeinduk info;
    struct listanak lanak;
    address next;
};
```



```

    address prev;
};

struct listinduk {
    address first;
    address last;
};

boolean ListEmpty (listinduk L);
boolean ListEmptyAnak (listanak L);

void CreateList (listinduk &L);
void CreateListAnak (listanak &L);

address alokasi (infotypeinduk X);
address_anak alokasiAnak (infotypeanak X);
void dealokasi (address P);
void dealokasiAnak (address_anak P);

address findElm (listinduk L, infotypeinduk X);
address_anak findElmAnak (listanak Lanak, infotypeanak X);
boolean fFindElm (listinduk L, address P);
boolean fFindElmAnak (listanak Lanak, address_anak P);
address findBefore (listinduk L, address P);
address_anak findBeforeAnak (listanak Lanak, infotypeinduk X,
address_anak P);

void insertFirst (listinduk &L, address P);
void insertAfter (listinduk &L, address P, address Prec);
void insertLast (listinduk &L, address P);
void insertFirstAnak (listanak &L, address_anak P);
void insertAfterAnak (listanak &L, address_anak P, address_anak
Prec);
void insertLastAnak (listanak &L, address_anak P);

void delFirst (listinduk &L, address &P);
void delLast (listinduk &L, address &P);
void delAfter (listinduk &L, address &P, address Prec);
void delP (listinduk &L, infotypeinduk X);

void delFirstAnak (listanak &L, address_anak &P);
void delLastAnak (listanak &L, address_anak &P);
void delAfterAnak (listanak &L, address_anak &P, address_anak

```

```

Prec);

void delPAanak (listanak &L, infotypeanak X);

void printInfo (listinduk L);
void printInfoAnak (listanak Lanak);
int nbList (listinduk L);
int nbListAnak (listanak Lanak);

void delAll (listinduk &L);

#endif

```

#multilist.cpp

```

#include "multilist.h"
#include <iostream>
using namespace std;

boolean ListEmpty(listinduk L) {
    return (L.first == Nil && L.last == Nil);
}

boolean ListEmptyAnak(listanak L) {
    return (L.first == Nil && L.last == Nil);
}

void CreateList(listinduk &L) {
    L.first = Nil;
    L.last = Nil;
}

void CreateListAnak(listanak &L) {
    L.first = Nil;
    L.last = Nil;
}

address alokasi(infotypeinduk X) {
    address P = new elemen_list_induk;
    P->info = X;
    P->lanak.first = Nil;
    P->lanak.last = Nil;
}

```

```

    P->next = Nil;
    P->prev = Nil;
    return P;
}

address_anak alokasiAnak(infotypeanak X) {
    address_anak P = new elemen_list_anak;
    P->info = X;
    P->next = Nil;
    P->prev = Nil;
    return P;
}

void dealokasi(address P) {
    delete P;
}

void dealokasiAnak(address_anak P) {
    delete P;
}

address findElm(listinduk L, infotypeinduk X) {
    address P = L.first;
    while (P != Nil && P->info != X) {
        P = P->next;
    }
    return P;
}

address_anak findElmAnak(listanak Lanak, infotypeanak X) {
    address_anak P = Lanak.first;
    while (P != Nil && P->info != X) {
        P = P->next;
    }
    return P;
}

void insertFirst(listinduk &L, address P) {
    if (ListEmpty(L)) {
        L.first = P;
        L.last = P;
    } else {
        P->next = L.first;
    }
}

```

```

        L.first->prev = P;
        L.first = P;
    }
}

void insertLast(listinduk &L, address P) {
    if (ListEmpty(L)) {
        insertFirst(L, P);
    } else {
        L.last->next = P;
        P->prev = L.last;
        L.last = P;
    }
}

void insertAfter(listinduk &L, address P, address Prec) {
    if (Prec != Nil) {
        P->next = Prec->next;
        P->prev = Prec;
        if (Prec->next != Nil) {
            Prec->next->prev = P;
        } else {
            L.last = P;
        }
        Prec->next = P;
    }
}

void insertFirstAnak(listanak &L, address_anak P) {
    if (ListEmptyAnak(L)) {
        L.first = P;
        L.last = P;
    } else {
        P->next = L.first;
        L.first->prev = P;
        L.first = P;
    }
}

void insertLastAnak(listanak &L, address_anak P) {
    if (ListEmptyAnak(L)) {
        insertFirstAnak(L, P);
    } else {

```

```

        L.last->next = P;
        P->prev = L.last;
        L.last = P;
    }
}

void insertAfterAnak(listanak &L, address_anak P, address_anak
Prec) {
    if (Prec != Nil) {
        P->next = Prec->next;
        P->prev = Prec;
        if (Prec->next != Nil) {
            Prec->next->prev = P;
        } else {
            L.last = P;
        }
        Prec->next = P;
    }
}

void delFirst(listinduk &L, address &P) {
    P = L.first;
    if (P != Nil) {
        if (L.first == L.last) {
            L.first = Nil;
            L.last = Nil;
        } else {
            L.first = P->next;
            L.first->prev = Nil;
        }
        P->next = Nil;
    }
}

void dellLast(listinduk &L, address &P) {
    P = L.last;
    if (P != Nil) {
        if (L.first == L.last) {
            L.first = Nil;
            L.last = Nil;
        } else {
            L.last = P->prev;
            L.last->next = Nil;
        }
    }
}

```

```

    }
    P->prev = Nil;
}
}

void delAfter(listinduk &L, address &P, address Prec) {
    if (Prec != Nil && Prec->next != Nil) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != Nil) {
            P->next->prev = Prec;
        } else {
            L.last = Prec;
        }
        P->next = Nil;
        P->prev = Nil;
    }
}

void delP(listinduk &L, infotypeinduk X) {
    address P = findElm(L, X);
    if (P != Nil) {
        if (P == L.first) {
            delFirst(L, P);
        } else if (P == L.last) {
            delLast(L, P);
        } else {
            address Q;
            Q = P;
            P->prev->next = P->next;
            P->next->prev = P->prev;
            Q->next = Nil;
            Q->prev = Nil;
        }
        dealokasi(P);
    }
}

void delFirstAnak(listanak &L, address_anak &P) {
    P = L.first;
    if (P != Nil) {
        if (L.first == L.last) {
            L.first = Nil;
        }
    }
}

```

```

        L.last = Nil;
    } else {
        L.first = P->next;
        L.first->prev = Nil;
    }
    P->next = Nil;
}

void dellLastAnak(listanak &L, address_anak &P) {
    P = L.last;
    if (P != Nil) {
        if (L.first == L.last) {
            L.first = Nil;
            L.last = Nil;
        } else {
            L.last = P->prev;
            L.last->next = Nil;
        }
        P->prev = Nil;
    }
}

void delAfterAnak(listanak &L, address_anak &P, address_anak Prec)
{
    if (Prec != Nil && Prec->next != Nil) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != Nil) {
            P->next->prev = Prec;
        } else {
            L.last = Prec;
        }
        P->next = Nil;
        P->prev = Nil;
    }
}

void delPAnak(listanak &L, infotypeanak X) {
    address_anak P = findElmAnak(L, X);
    if (P != Nil) {
        if (P == L.first) {
            delFirstAnak(L, P);

```

```

        } else if (P == L.last) {
            delLastAnak(L, P);
        } else {
            P->prev->next = P->next;
            P->next->prev = P->prev;
            P->next = Nil;
            P->prev = Nil;
        }
        dealokasiAnak(P);
    }
}

void printInfo(listinduk L) {
    address P = L.first;
    while (P != Nil) {
        cout << "Induk: " << P->info << endl;
        printInfoAnak(P->lanak);
        P = P->next;
    }
}

void printInfoAnak(listanak Lanak) {
    address_anak P = Lanak.first;
    while (P != Nil) {
        cout << "  Anak: " << P->info << endl;
        P = P->next;
    }
}

void delAll(listinduk &L) {
    address P;
    while (!ListEmpty(L)) {
        delFirst(L, P);
        dealokasi(P);
    }
}

```

#main.cpp

```

#include <iostream>
#include "multilist.h"

```



```

#include "multilist.cpp"
using namespace std;

int main() {
    listinduk L;
    CreateList(L);

    address A = alokasi(10);
    address B = alokasi(20);
    address C = alokasi(30);

    insertFirst(L, A);
    insertLast(L, B);
    insertLast(L, C);

    insertFirstAnak(A->lanak, alokasiAnak(1001));
    insertLastAnak(A->lanak, alokasiAnak(1002));

    insertFirstAnak(B->lanak, alokasiAnak(2001));

    cout << "Data Multilist:\n";
    printInfo(L);

    cout << "\nMenghapus anak 1002...\n";
    delPAnak(A->lanak, 1002);
    printInfo(L);

    cout << "\nMenghapus induk 20...\n";
    delP(L, 20);
    printInfo(L);

    return 0;
}

```

Screenshots Output

```
Data Multilist:
Induk: 10
  Anak: 1001
  Anak: 1002
Induk: 20
  Anak: 2001
Induk: 30

Menghapus anak 1002...
Induk: 10
  Anak: 1001
Induk: 20
  Anak: 2001
Induk: 30

Menghapus induk 20...
Induk: 10
  Anak: 1001
Induk: 30

[Done] exited with code=0 in 0.822 seconds
```

Deskripsi :

Program di atas merupakan implementasi struktur data multilist yang terdiri atas dua jenis daftar, yaitu daftar induk dan daftar anak, di mana setiap elemen induk dapat memiliki daftar anak masing-masing. Program ini menyediakan berbagai fungsi dasar seperti membuat list baru, mengecek apakah list kosong, menambah elemen di awal, akhir, atau setelah elemen tertentu, serta menghapus elemen berdasarkan posisi maupun berdasarkan nilai. Selain itu, program juga menyediakan fungsi pencarian elemen, menampilkan seluruh isi list beserta anak-anaknya, dan menghapus seluruh elemen yang ada. Seluruh operasi dilakukan menggunakan konsep doubly linked list, sehingga tiap elemen terhubung dua arah untuk memudahkan proses penambahan dan penghapusan elemen tanpa harus menggeser struktur data lain. Program ini berguna untuk merepresentasikan data yang memiliki hubungan hierarki, misalnya kategori dan sub-kategori, atau bagian dan sub-bagian dalam suatu sistem.

#Unguided 2

#circularlist.h

```
#ifndef CIRCULARLIST_H_INCLUDED
#define CIRCULARLIST_H_INCLUDED

#include <iostream>
#include <string>
using namespace std;

struct mahasiswa {
    string nama;
    string nim;
    char jenis_kelamin;
    float ipk;
};

typedef mahasiswa infotype;

typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void insertFirst(List &L, address P);
void insertAfter(List &L, address Prec, address P);
void insertLast(List &L, address P);
void deleteFirst(List &L, address &P);
void deleteAfter(List &L, address Prec, address &P);
void deleteLast(List &L, address &P);
address findElm(List L, string nim);
void printInfo(List L);
```

```
#endif
```

#circularlist.cpp

```
#include "circularlist.h"

void CreateList(List &L) {
    L.First = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    if (P != NULL) {
        P->info = x;
        P->next = P; // circular
    }
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void insertFirst(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        P->next = P;
    } else {
        address Q = L.First;
        while (Q->next != L.First) Q = Q->next;
        Q->next = P;
        P->next = L.First;
        L.First = P;
    }
}

void insertAfter(List &L, address Prec, address P) {
    if (Prec != NULL) {
        P->next = Prec->next;
        Prec->next = P;
    }
}
```

```

void insertLast(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        P->next = P;
    } else {
        address Q = L.First;
        while (Q->next != L.First) Q = Q->next;
        Q->next = P;
        P->next = L.First;
    }
}

void deleteFirst(List &L, address &P) {
    if (L.First != NULL) {
        P = L.First;
        if (P->next == P) {
            L.First = NULL;
        } else {
            address Q = L.First;
            while (Q->next != L.First) Q = Q->next;
            L.First = P->next;
            Q->next = L.First;
        }
        P->next = NULL;
    }
}

void deleteAfter(List &L, address Prec, address &P) {
    if (Prec != NULL && Prec->next != NULL) {
        P = Prec->next;
        Prec->next = P->next;
        P->next = NULL;
    }
}

void deleteLast(List &L, address &P) {
    if (L.First != NULL) {
        address Q = L.First;
        if (Q->next == Q) {
            P = Q;
            L.First = NULL;
        } else {

```

```

        while (Q->next->next != L.First) Q = Q->next;
        P = Q->next;
        Q->next = L.First;
    }
    P->next = NULL;
}

address findElm(List L, string nim) {
    if (L.First == NULL) return NULL;
    address P = L.First;
    do {
        if (P->info.nim == nim) return P;
        P = P->next;
    } while (P != L.First);
    return NULL;
}

void printInfo(List L) {
    if (L.First == NULL) {
        cout << "List kosong" << endl;
        return;
    }
    address P = L.First;
    do {
        cout << P->info.nama << " | " << P->info.nim << " | "
            << P->info.jenis_kelamin << " | " << P->info.ipk <<
endl;
        P = P->next;
    } while (P != L.First);
}

```

#main.cpp

```

#include "circularlist.h"
#include "circularlist.cpp"
#include <iostream>
using namespace std;

address createData(string nama, string nim, char jenis_kelamin,
float ipk)

```

```

{
    infotype x;
    address P;
    x.nama = nama;
    x.nim = nim;
    x.jenis_kelamin = jenis_kelamin;
    x.ipk = ipk;
    P = alokasi(x);
    return P;
}

int main()
{
    List L, A, B, L2;
    address P1 = NULL;
    address P2 = NULL;
    infotype x;

    CreateList(L);
    cout << "coba insert first, last, dan after" << endl;

    P1 = createData("Danu", "04", 'l', 4.0);
    insertFirst(L, P1);

    P1 = createData("Fahmi", "06", 'l', 3.45);
    insertLast(L, P1);

    P1 = createData("Bobi", "02", 'l', 3.71);
    insertFirst(L, P1);

    P1 = createData("Ali", "01", 'l', 3.3);
    insertFirst(L, P1);

    P1 = createData("Gita", "07", 'p', 3.75);
    insertLast(L, P1);

    x.nim = "07";
    P1 = findElm(L, x.nim);
    P2 = createData("Cindi", "03", 'p', 3.5);
    insertAfter(L, P1, P2);

    x.nim = "02";
    P1 = findElm(L, x.nim);

```

```

    P2 = createData("Hilmi", "08", 'p', 3.3);
    insertAfter(L, P1, P2);

    x.nim = "04";
    P1 = findElm(L, x.nim);
    P2 = createData("Eli", "05", 'p', 3.4);
    insertAfter(L, P1, P2);

    printInfo(L);

    return 0;
}

```

Screenshots Output

```

coba insert first, last, dan after
Ali | 01 | l | 3.3
Bobi | 02 | l | 3.71
Hilmi | 08 | p | 3.3
Danu | 04 | l | 4
Eli | 05 | p | 3.4
Fahmi | 06 | l | 3.45
Gita | 07 | p | 3.75
Cindi | 03 | p | 3.5

[Done] exited with code=0 in 0.932 seconds

```

Deskripsi:

Program ini dibuat untuk mengelola data mahasiswa menggunakan struktur Circular Linked List, yaitu daftar berantai yang elemen terakhirnya selalu terhubung kembali ke elemen pertama. Setiap elemen menyimpan informasi mahasiswa berupa nama, NIM, jenis kelamin, dan IPK. Program menyediakan berbagai operasi seperti membuat list baru, menambah data di awal, di akhir, maupun setelah elemen tertentu, mencari data berdasarkan NIM, serta menampilkan seluruh data yang tersimpan. Dalam fungsi main, program membuat list kemudian menambahkan beberapa data mahasiswa menggunakan createData, insertFirst, insertLast, dan insertAfter, dengan bantuan findElm untuk menemukan posisi penyisipan yang tepat. Setelah seluruh data dimasukkan, program menampilkan isi list menggunakan printInfo. Program ini menunjukkan bagaimana Circular Linked List dapat digunakan untuk menyimpan dan mengatur data secara terstruktur serta menampilkannya dengan mudah.

D. Kesimpulan

Secara keseluruhan, praktikum struktur data mulai dari pembuatan Circular Linked List hingga implementasi Multilist memberikan pemahaman yang utuh tentang cara mengelola data dinamis yang saling terhubung. Melalui rangkaian program yang dibuat, kita belajar melakukan alokasi dan dealokasi memori, menambah dan menghapus elemen pada berbagai posisi, mencari data tertentu, serta menampilkan isi list secara lengkap. Pada Circular Linked List, kita memahami konsep list melingkar di mana node terakhir selalu kembali ke node pertama, sedangkan pada Multilist kita mempelajari struktur hierarki yang menghubungkan data induk dan data anak sehingga lebih fleksibel untuk merepresentasikan data bertingkat. Seluruh proses ini membantu kita memahami cara kerja pointer, hubungan antar-node, serta pengelolaan struktur data yang lebih kompleks untuk menyelesaikan berbagai permasalahan dalam pemrograman.

E. Referensi

Butler, Jamie; Hoglund, Greg. ["VICE – Catch the hookers! \(Plus new rootkit techniques\)"](#) (PDF). Archived from [the original](#) (PDF) on 2016-10-01. Retrieved 2021-08-31.

Collins, William J. (2005) [2002]. *Data Structures and the Java Collections Framework*. New York: McGraw Hill. pp. 239–303. [ISBN 0-07-282379-8](#).

Juan, Angel (2006). ["Ch20 –Data Structures: ID06 - PROGRAMMING with JAVA \(slide part of the book 'Big Java', by CayS. Horstmann\)"](#) (PDF). p. 3. Archived from [the original](#) (PDF) on 2012-01-06. Retrieved 2011-07-10.